

INTRODUÇÃO À PROGRAMAÇÃO

Parte 2

INTRODUÇÃO ÀS FUNÇÕES

```
#include <stdio.h>
mensagem()
{
    printf( "Olá" );
}
main()
{
    mensagem();
    printf( "Boa tarde. \n" );
}
```

Em C, todas as funções têm de devolver um valor.
Para tal, neste exemplo, temos de utilizar o tipo **VOID**.

```
void mensagem(void)
{
    printf( "Olá" );
}
```

PARÂMETROS (ARGUMENTOS) DE FUNÇÕES

- ❑ **Passagem por valor:** se existir uma alteração dentro da função o valor não é afectado fora.

```
#include <stdio.h>
void quadrado(int x)
{
    printf(" O quadrado é %d",(x*x));
}
main()
{
    int num;
    printf("Int. um numero: ");
    scanf("%d",&num);
    printf("\n\n");
    quadrado(num);
}
```

EXEMPLO

```
#include <stdio.h>

void multiplica(float a, float b, float c)
{
    printf("%.2f",a*b*c);
}

main()
{
    float x,y;
    x = 23.5;
    y = 12.9;
    multiplicar(x,y,3.87);
}
```

DEVOLVER VALORES: *RETURN*

```
#include <stdio.h>
float produto(int x, int y)
{
    return(x*y);
}
main()
{
    int valor;
    valor = produto(5,4);
    printf( ' 'O resultado e : %d\n' ', valor);
}
```


VARIÁVEIS LOCAIS E GLOBAIS

```
#include <stdio.h>
```

```
int valor1, valor2, valor3;
```

```
int soma(void)
```

```
{
```

```
    int resultado;
```

```
    resultado = valor1 + valor2 + valor3;
```

```
    return resultado;
```

```
}
```

```
main()
```

```
{
```

```
    int resultado;
```

```
    valor1 = 10; valor2 = 20; valor3 = 30;
```

```
    resultado = soma();
```

```
    printf( ' 'A soma de %d+%d+%d = d\n' ',valor1,valor2,valor3,resultado);
```

```
}
```

Variáveis globais

Variável local

Variável local

VARIÁVEIS LOCAIS E GLOBAIS

```
int n1;  
void funcao1(void)  
{  
    n1 = 10;  
    n2 = 20;  
}  
int n2, n3;  
void funcao2(void)  
{  
    n2 = 20;  
    n3 = 30;  
}
```

Variável ainda não declarada



STRING'S

- ❑ No C, a *string* é tratada como sendo um vector de caracteres:

char nome_da_string[tamanho]

J	O	S	E	\0				
---	---	---	---	----	--	--	--	--

char string[50]

- ❑ O último elemento é `'\0'`

```
char string[50];  
int i;  
for (i=0;string[i];i++) printf( ' '%c' ',string[i]);
```
- ❑ Podemos utilizar a função ***gets(nome_var)*** para ler do teclado uma *string*.
- ❑ Para escrever podemos utilizar ***puts(nome_var)***
- ❑ É válido efectuar ***printf(string);***

STRING'S

- ❑ A função ***scanf*** também permite a leitura de uma string utilizando o formato ***%s***.
- ❑ No entanto, a variável que recebe a *string* NÃO É PRECEDIDA de um ***&***.
- ❑ ***scanf("%s", nomevar_string);***
- ❑ A função ***scanf*** apenas faz a leitura de uma palavra.

STRING'S

- ❑ Não devemos atribuir o valor de uma *string* a outra *string*:

```
string2 = string1; /* ERRADO */
```
- ❑ Para as diversas operações com *strings* utilizar as funções da biblioteca **string.h**.
- ❑ **strcpy(string2,string1);** copia a *string1* para a *string2*.
- ❑ **strcat(str_destino,str_origem);** é acrescentada a *str_origem* no final da *str_destino*.
- ❑ **strlen(string);** comprimento da *string* (o caracter `'\0'` não é contado).
- ❑ **strcmp(string1,string2);** 0 (zero) se iguais e 1 se diferentes.

EXEMPLO

```
#include <stdio.h>
#include <string.h>
main()
{
    char s1[20],s2[20],s3[20];
    printf(´´ Introduza uma string : ´´);
    gets(s1);
    strcpy(s2,s1);
    strcpy(s3,´´ A string introduzida é : ´´);
    printf(´´\n %s %s \n ´´ , s3, s2);
}
```

float x [20]

- ❑ Começa sempre em zero;
- ❑ É reservado o espaço necessário para a variável
 - ❑ Neste caso: 4bytes x 20 = 80 bytes
- ❑ Dados indexados de x[0] a x[19]
- ❑ ATENÇÃO!!! Os índices não são validados

PASSAGEM DE VECTORES PARA FUNÇÕES

```
main()
{   int v[10];
    int x[20];

    inic1(v);
    inic2(x);
}
```

int [10]

```
void inic1(int s[10])
{
    int i;
    for(i=0;i<10;i++) s[i]=0;
}
```

int [20]

```
void inic2(int s[20])
{
    int i;
    for(i=0;i<20;i++) s[i]=0;
}
```

PASSAGEM DE VECTORES PARA FUNÇÕES

```
main()
{   int v[10];
    int x[20];

    inic1(v,10);
    inic2(x,20);
}
```

```
void inic(int s[ ], int n)
{
    int i;
    for(i=0;i<n;i++) s[i]=0;
}
```

MATRIZES BIDIMENSIONAIS

tipo nome[linhas][colunas];

❑ Matrizes de *string*'s

char string[5][50];

```
for (i=0;i<5;i++)  
{  
    printf( ' ' Introduza uma string : ' ' );  
    gets(string[i]);  
}  
for (i=0;i<5;i++) printf( ' '%s \n ' ', string[i]);
```

INICIALIZAÇÃO

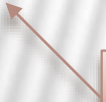
```
float x[6] = {1.3,4.5,2.7,4.1,0.0,50.3};  
int mat[2][5] = {1,2,3,4,5,6,7,8,9,10};  
char str[10] = {'J','o','s','é','\0'};  
char str[10] = "José";  
char strnomes[3][10] = {"José", "Ana", "Maria"};
```

35



```
char mensg[] = "Linguagem C: flexibilidade e poder.";  
int mat[][2] = {1,2,3,4,5,6,7,8,9,10};
```

5





IPG

Politécnico
da Guarda

Escola Superior
de Tecnologia e Gestão

PONTEIROS

PONTEIROS

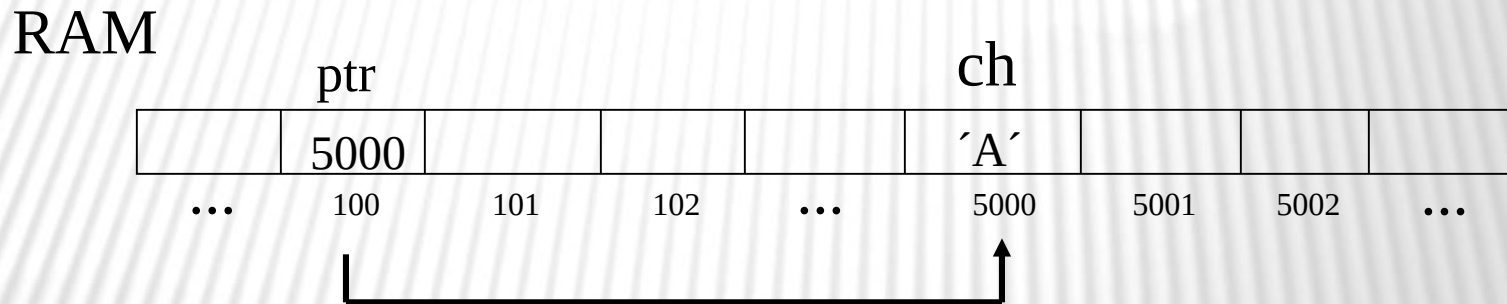
❑ Declaração

*tipo do ponteiro * nome_da_variável*

- ❑ *int *pt*; o compilador sabe que a variável *pt*, não vai guardar um valor, mas sim **um endereço**.
- ❑ Um ponteiro é uma variável que aponta sempre para outra variável de determinado tipo.

PONTEIROS

- Sempre que declaramos uma variável, temos de indicar qual o seu tipo e o seu nome: ***char ch;***



- Quando se faz: ***ch = 'A';***
- Declaração de um ponteiro : ***char *ptr;***
- Para colocar ***ptr*** a apontar para a variável ***ch***: ***ptr = &ch;***

PONTEIROS

- Um bom hábito para evitar problemas de programação é a inicialização dos ponteiros:

```
int i = 5;
```

```
float x = 10.8;
```

```
int * iptr = &i;
```

```
float * xptr = &x;
```

- No entanto podem existir situações em que posso declarar um ponteiro e não querer que este aponte para variável alguma. Neste caso posso colocá-lo a apontar para **NULL**.

```
int * ptr = NULL;
```

EXEMPLO

```
#include <stdio.h>
main()
{
    char *temp, *pt2;
    int count = 10, x, *iptr;

    iptr = &count;
    x = *iptr;
    printf( ' 'Valor de x = %d \n ' ',x);
    printf( ' 'Endereço do ponteiro %p \n ' ',iptr);
    printf( ' 'Valor da variável apontada por iptr %d \n ' ',*iptr);
}
```

Output:

10
(valor do endereço)
10

Atribui o endereço de *count* a *iptr*

Atribui o valor guardado pelo
endereço especificado pelo ponteiro *iptr*

EXEMPLO

...

```
char c = 'Q';  
char *cptr = &c;
```

```
printf(' ' '%c - %c \n' ' ',c,*cptr);  
c = 'Z';  
printf(' ' '%c - %c \n' ' ',c,*cptr);  
*cptr = 'Y';  
printf(' ' '%c - %c \n' ' ',c,*cptr);
```

...

Output:

Q - Q
Z - Z
Y - Y

EXEMPLO

```
...  
int i1,i2,*p1,*p2;  
  
i1 = 5;  
p1 = &i1;  
i2 = *p1/2 + 10;  
p2 = p1;  
printf( ' 'i1 = %d,\n i2 = %d,  
        \n *p1 = %d,\n *p2 = %d\n ' ',i1,i2,*p1,*p2);  
...
```

Output:

```
i1 = 5  
i2 = 12  
*p1 = 5  
*p2 = 5
```

OPERAÇÕES COM PONTEIROS

*int *p;*

p++;

← Aponta para os próximos dois bytes

*(*p)++;*

← Incrementa o conteúdo da variável apontada por p

p = p + 10;

← Aponta para 10 posições mais à frente

**(p + 10);*

← Valor da posição 10 posições à frente

RESUMO DAS OPERAÇÕES COM PONTEIROS

Operação	Exemplo	Observações
Atribuição	<code>ptr = &x</code>	Podemos atribuir um valor (endereço) a um ponteiro. Se quisermos que aponte para nada podemos atribuir-lhe o valor da constante NULL
Incremento	<code>ptr = ptr+2</code>	Incremento de $2 * \text{sizeof}(\text{tipo})$ de <code>ptr</code> .
Decremento	<code>ptr = ptr -10</code>	Decremento de $10 * \text{sizeof}(\text{tipo})$ de <code>ptr</code> .
Apontado por	<code>*ptr</code>	O operador asterisco permite obter o valor existente na posição cujo endereço está armazenado em <code>ptr</code> .
Endereço de	<code>&ptr</code>	Tal como qualquer outra variável, um ponteiro ocupa espaço em memória. Desta forma podemos saber qual o endereço que um ponteiro ocupa em memória.
Diferença	<code>ptr1 - ptr2</code>	Permite-nos saber o número de elementos entre <code>ptr1</code> e <code>ptr2</code> .
Comparação	<code>ptr1 > ptr2</code>	Permite-nos verificar, por exemplo, qual a ordem de dois elementos num vector, através dos seus endereços.

MATRIZES E PONTEIROS

int x[3];

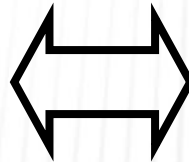
variável[indice] $\Rightarrow$ x[2]

*(variável + índice) $\Rightarrow$ *(p+2)

EXEMPLO

```
main()
{
    int mat[10][10];
    int i,j;

    for (i=0;i<10;i++)
        for (j=0;j<10;j++)
            mat[i][j] = 0;
}
```



```
main()
{
    int mat[10][10];
    int *p, i;

    p = mat[0];
    for (i=0;i<(10*10);i++)
    {
        *p = 0;
        p++;
    }
}
```


CONSIDERAÇÕES GERAIS

```
for (i=0, total =0;i<=10;i++)  
    total = total + i;
```

```
for (i=0, j=0;i<=10;i++,j--)  
    printf( ' '%d - %d\n' ',i,j);
```

CONSIDERAÇÕES GERAIS

```
StrCpy(char *destino, char *origem)
{
    while (*origem)
    {
        *destino = *origem;
        origem++;
        destino++;
    }
    *destino = '\0';
}
```

CONSIDERAÇÕES GERAIS

- ❑

```
while ((n=scanf( ' ' %d%d ' ',&i,&j))==2)
{
    printf( ' ' %d\n ' ',(i+j));
    n_pares++;
}
if (n != EOF) printf( ' ' Erro !!!\n ' ' );
```

- ❑

```
for (n_pares=1;(n=scanf( ' ' %d%d ' ',&i,&j))==2);n_pares++);
```

PASSAGEM POR VALOR

```
#include <stdio.h>

void troca(int a, int b);

main()
{
    int n1,n2;
    n1 = 100;
    n2 = 200;
    troca(n1,n2);
    printf( ' '%d %d\n' ',n1,n2);
}
```

```
void troca(int a,int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}
```

Output:
100 200

Passagem por valor:
o valor passado para a função
não retorna.

PASSAGEM POR REFERÊNCIA

```
#include <stdio.h>

void troca(int *a, int *b);

main()
{
    int n1,n2;
    n1 = 100;
    n2 = 200;
    troca(&n1,&n2);
    printf( ' '%d%d\n' ',n1,n2);
}
```

```
void troca(int *a,int *b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```

Output:
200 100

Passagem por referência:
os valores são alterados.

PASSAGEM DE VECTORES PARA FUNÇÕES

```
void funcao1(int x[10], int n)
```

```
{  ...  
    x[i]
```

```
}
```

```
void funcao2(int x[],int n)
```

```
{  ...  
    x[i]
```

```
}
```

```
void funcao3(int *p, int n)
```

```
{      ou  p[i]
```

```
    *p
```

```
    p++
```

```
}
```

```
main()  
{
```

```
    int vector[10];
```

```
    int dim = 5;
```

```
    ...
```

```
    funcao1(vector,dim);
```

```
    funcao2(vector,dim);
```

```
    funcao3(vector,dim);
```

```
    ...
```

```
}
```

PASSAGEM DE MATRIZES PARA FUNÇÕES

```
void funcao4(int x[MAX][MAX], int n,int m)
```

```
{  ...  
  x[i][j]  
}
```

```
void funcao5(int x[][MAX], int n,int m)
```

```
{  ...  
  x[i][j]  
}
```

```
void funcao6(int *p, int n,int m)
```

```
{  
  *p  
  p++  
}
```

```
main()  
{
```

```
  int tabela[MAX][MAX];  
  int diml=5, dimc=2;
```

```
  ...  
  funcao4(tabela,diml,dimc);  
  funcao5(tabela,diml,dimc);  
  funcao6(&tabela[0][0],diml,dimc);
```

```
      ou  
  funcao6(tabela[0],diml,dimc);
```

```
  ...  
}
```

PASSAGEM DE MATRIZES PARA FUNÇÕES

```
void funcao7(int *p, int n,int m)
{
    *(p + i * MAX +j)
}
```

```
main()
{
    ...
    funcao7(&tabela[0][0],diml,dimc);
    ...
}
```

```
int tabela[MAX][MAX];
int diml=5, dimc=2;
...
funcao7(&tabela[0][0],diml,dimc);
...
```

OUTROS EXEMPLOS

```
int **cria_matriz(int linhas, int colunas)
{
    int i;
    int **M;

    M = (int**) malloc(sizeof(int*)*linhas);
    for (i=0;i<linhas;i++)
        M[i] = (int*) malloc(sizeof(int)*colunas);

    return M;
}
```

OUTROS EXEMPLOS

```
void apaga_matriz(int linhas, int **M)
{
    int i;
    for(i=0;i<linhas;i++)
        free(M[i]);

    free(M);
}
```


OUTROS EXEMPLOS

```
void funcao8(int **p, int n, int m)
{
    p[i][j]
}
```

```
main()
{
    int **t;
    int diml=5,dimc=2;

    t = cria_matriz(diml,dimc);
    funcao8(t,diml,dimc);
    ...
    funcao1(t[0],dimc);
    funcao3(t[0],dimc);
    ...
    apaga_matriz(diml,t);
}
```



```
#include <stdio.h>
#define MAX_ARRAY 100
```

```
main()
{
    int list[MAX_ARRAY], j, sort_num;

    printf("Quantos valores vai introduzir ? ");
    scanf("%d", &sort_num);

    if (sort_num > MAX_ARRAY )
    {
        printf("Demasiados valores ... o máximo é %d \n",MAX_ARRAY);
        sort_num = MAX_ARRAY;
    }
    for (j=0; j < sort_num; j++)
        scanf("%d", &list[j]);
}
```



```
#include <stdio.h>
#include <stdlib.h>
```

```
main()
{
    int *list, sort_num, j;

    printf("Quantos elementos deseja introduzir : ");
    scanf("%d",&sort_num);

    list = (int *) malloc (sort_num * sizeof(int) );

    for (j = 0;j<sort_num; j++)
        scanf("%d",list + j );

}
```

PROTÓTIPOS DE FUNÇÕES

tipo nomefunção(parametros);

Protótipo da função

main()

{

}

tipo nomefunção(parametros)

{

Corpo da função

}

PROTÓTIPOS DE FUNÇÕES

```
int maximo( int [] ); /* Protótipo da função */
```

```
main()  
{  
    int valores[5];  
    ...  
    maximo(valores);  
}  
int maximo(int val[5]);  
{  
    ...  
}
```


PROTÓTIPOS DE FUNÇÕES

```
int matriz( int[][5], int ); /* Protótipo da função */
```

```
main()  
{  
    int num[2][5];  
    ...  
    matriz(num,2);  
}  
int matriz(int array[][5], int linhas);  
{  
    ...  
}
```

RECURSIVIDADE

- Em linguagem C, uma função pode ser chamada de um modo recursivo, isto é, uma função pode chamar-se a si própria.

CARACTERÍSTICAS DAS FUNÇÕES RECURSIVAS

□ As funções recursivas contêm duas partes fundamentais:

1. **Ponto de Paragem:** o ponto de paragem da recursividade é resolvido sem utilização de recursividade, sendo este ponto geralmente um limite superior ou inferior da regra geral.
2. **Regra Geral:** o método geral da recursividade reduz a resolução do problema através da invocação recursiva de casos particulares até atingir o ponto de paragem que finaliza o método.

CARACTERÍSTICAS DAS FUNÇÕES RECURSIVAS

□ Exemplo:

$$Factorial(n) = \begin{cases} (n == 0) \rightarrow 1 & \text{Ponto de Paragem} \\ (n) \rightarrow n * (n - 1)! & \text{Regra Geral} \end{cases}$$

RECURSIVIDADE

- ❑ Exercícios propostos:
 - ❑ Cálculo Factorial
 - ❑ Somatório $\sum_{i=1}^n X_i$
 - ❑ Cálculo Fibonacci
 - ❑ Calcular o número de dias decorridos até uma data (dia/mês/ano)

DIRECTIVAS DE COMPILAÇÃO

- ❑ Comandos não compilados
- ❑ Dirigidos ao compilador

Exemplo:

- ❑ `#include ´´nome_do_arquivo´´`
 - ❑ Para indicar o caminho completo ou utilizar o directório corrente;
- ❑ `#include <nome_do_arquivo>`
 - ❑ Directórios usuais utilizados pelo compilador.

As macros são porções de código que são substituídas, pelo Pré-Processador, antes mesmo do compilador passar pelo código.

```
#define MAX 100
```

Instrução dirigida ao Pré-Processador para que este, a partir desta linha, substitua todas as ocorrências de MAX pelo valor 100.

O termo **MACRO** não é normalmente associada à definição de constantes simbólicas. Normalmente o termo macro, refere-se a chamadas a funções que são substituídas por outro código pelo Pré-Processador.

#DEFINE

`#define nome_da_macro seq_de_caracteres`

Exemplos:

`#define PI 3.1416`

`#define Versao ``2.00```

`#define TRUE 1`

`#define FALSE 0`

`#define OR ||`

`#define EQUALS ==`

EXEMPLOS

```
#define OLA printf( ' 'Ola !!! \n ' ');  
#define OLA(expr) printf( #expr ' '=%g/n ' ',expr);  
Chamada : OLA(3/4)  $\Rightarrow 3/4 = 0.75$ 
```

```
#define FOREVER for(;;)  
#define QUADRADO(x) x*x
```

OPERADOR DE CONTINUIDADE

Cada macro tem de ser escrita numa única linha.

```
#define Expressao(x,y,z) ((x)*2+3*(y)-1/(z))
```

Se existir a necessidade de escrever uma macro por várias linhas, deve-se colocar o operador de continuidade das macros \ no final de cada linha a continuar.

```
#define Expressao(x,y,z) ((x)*2 + \  
                        3*(y) - \  
                        1/(z))
```


COLOCAÇÃO DE PARÊNTESES

#define Multiplica(x,y) ((x)(y))*

Quando se utiliza : *Multiplica(3+1,2+3)*

A expansão da macro é feita colocando

(primeiro parâmetro) * (segundo parâmetro)

Sem parêntesis: $3 + 1 * 2 + 3$

$$3 + (1 * 2) + 3 = 8$$

O Resultado deveria ser :

Com parêntesis: $(3 + 1) * (2 + 3)$

$$4 * 5 = 20$$

COMPILAÇÃO CONDICIONAL

#if ... #endif

O Pré-processador pode retirar algum código da compilação, compilar partes na situação x e compilar outras na situação y, através do seguinte conjunto de directivas:

```
#if condição_1
    instruções_1
#elif condição_2
    instruções_2
...
#else
    instruções_por_defeito
#endif
```

#IF - #ELIF - #ELSE - #ENDIF

```
#if SYSTEM == SYSV
    #define HDR "sysv.h"
#elif SYSTEM == BSD
    #define HDR "bsd.h"
#elif SYSTEM == MSDOS
    #define HDR "msdos.h"
#else
    #define HDR "default.h"
#endif
#include HDR
```

OPERADOR DEFINED()

```
#include <stdio.h>
```

```
#include "def.h" → def.h
```

```
main()
```

```
{
```

```
    printf("%d\n",QUADRADO(2));
```

```
}
```

```
#if !defined(DEF)
```

```
    #define DEF
```

```
        #define PI 3.1416
```

```
        #define Versao ``2.00``
```

```
        #define TRUE 1
```

```
        #define FALSE 0
```

```
        #define OR ||
```

```
        #define EQUALS ==
```

```
        #define OLA printf( ``Ola !!! \n`` );
```

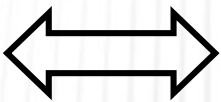
```
        #define OLA(expr) printf( #expr ``= %g/n`` ,expr );
```

```
        #define FOREVER for(;;)
```

```
        #define QUADRADO(x) x*x
```

```
#endif
```

#IFDEF - #ENDIF

`#ifndef HDR`  `#if !defined(HDR)`
 `#define HDR`
 conteúdo de hdr.h
`#endif`

DECLARAÇÃO DE ESTRUTURAS (REGISTOS)

```
struct [nome_estrutura]
{
    tipo1 campo1, campo2;
    ...
    tipon campo;
};
```

DECLARAÇÃO DE ESTRUTURAS (REGISTOS)

```
struct ponto {  
    int x;  
    int y;  
};
```

```
struct ponto pt;  
struct ponto maxpt = {320,200};
```

```
printf( ' '%d,%d' ',pt.x,pt.y);
```

```
struct rectangulo {  
    struct ponto pt1;  
    struct ponto pt2;  
};
```

```
struct rectangulo ecran;
```

```
ecran.pt1.x, ecran.pt1.y  
ecran.pt2.x, ecran.pt2.y
```

REGISTOS

```
struct ponto criaponto(int x, int y) {  
    struct ponto temp;  
  
    temp.x = x;  
    temp.y = y;  
  
    return temp;  
}  
  
struct ponto somaponto(  
    struct ponto p1,  
    struct ponto p2)  
{  
    p1.x + = p2.x;  
    p1.y + = p2.y;  
  
    return p1;  
}
```

```
struct rectangulo ecran;  
struct ponto medio;  
struct ponto criaponto(int, int);  
ecran.pt1 = criaponto(0,0);  
ecran.pt2 = criaponto(XMAX, YMAX);  
medio=criaponto((ecran.pt1.x+ecran.pt2.x)/2,(ecran.pt1.y+ecran.pt2.y)/2);  
  
int ptinrect(struct ponto p, struct rectangulo r)  
{  
    return p.x >=r.pt1.x && p.x < r.pt2.x && p.y >= r.pt1.y && p.y < r.pt2.y;  
}
```


TYPEDEF

- ❑ Uma das desvantagens na utilização de estruturas está na declaração das variáveis que têm sempre que ser precedidas da palavra reservada **struct** seguida do nome da estrutura.
- ❑ O ideal seria podermos representar uma estrutura unicamente através de uma palavra (um sinónimo), tal como fazemos com os tipos base da linguagem.
- ❑ Isso é possível através da palavra reservada **typedef**:

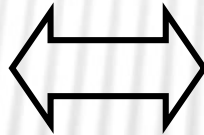
typedef tipo_existente sinónimo

TYPEDEF

□ Exemplos:

```
typedef int Inteiro;  
Inteiro x, y;
```

```
typedef struct dados{  
    char nome[50];  
    int idade;  
    float salario;  
} infor;
```



infor funcionario;

...

funcionario.nome
funcionario.idade
funcionario.salario

infor pessoas[100];
pessoas[50].salario
pessoas[2].nome

PONTEIROS E REGISTOS

```
struct ponto *pp;           ⇒      (*pp).x
                                (*pp).y
```

- Uma alternativa à notação anterior é a utilização do $\dot{\rightarrow}$.

$(*pp).x \Rightarrow pp \rightarrow x$

```
struct rectangulo r, *rp = &r;
```

	r.pt1.x
	rp->pt1.x
	(r.pt1).x
	(rp->pt1).x

UNION

- ❑ Uma variável do tipo *union* pode conter (em instantes diferentes) valores de diferentes tipos de tamanhos.

```
union valor {  
    int    ival;  
    float  fval;  
    char   *sval;  
}
```

Quando se preenche um dos campos os outros são automaticamente destruídos.

Os vários campos tem o mesmo endereço de memória.

UNION

- ❑ Em cada momento só pode existir uma e uma só variável.

Utilização:

- ❑ u.ival

- ❑ union valor tipos;

⇒ tipos.fval

ENUM

- É uma forma eficiente de associar nomes de constantes a valores, é uma alternativa ao *#define*.

```
enum meses { jan = 1, fev, mar, abr, mai, jun, jul,  
            ago, set, out, nov, dez};
```

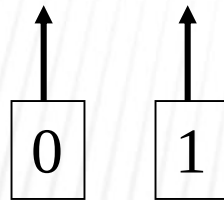
Normalmente inicia em 0

```
enum meses este_mes;
```

```
este_mes = abr;
```

ENUM

```
enum boolean { NO,YES };
```



- Também é possível associar outros tipos de valores, como por exemplo:

```
enum escapes { BELL= '\a', BACKSPACE = '\b',  
               TAB = '\t',NEWLINE = '\n'};
```

AUTO E EXTERN

- ❑ **auto**: especifica que a variável é local. Raramente é utilizada porque todas as variáveis são *auto* por definição. Existem e desaparecem com a chamada e termino das funções.
- ❑ Uma alternativa é a utilização da declaração **extern**, onde as variáveis podem ser utilizadas por todas as funções.

AUTO E EXTERN

```
/* Exemplo1.h */
```

```
int x = 2;
```

```
int funcao1()
```

```
{
```

```
    return x * 2;
```

```
}
```

```
/* Exemplo1.c */
```

```
#include <stdio.h>
```

```
#include "exemplo1.h"
```

```
int x;
```

```
void verfuncao()
```

```
{
```

```
    printf("%d\n",x);
```

```
}
```

```
main()
```

```
{
```

```
    x = funcao1();
```

```
    verfuncao();
```

```
}
```

O compilador vai dizer que a variável foi declarada mais de uma vez. Desta forma, temos de utilizar a declaração:

```
extern int x;
```

AUTO E STATIC

- As variáveis **static** são criadas e inicializadas apenas uma vez.

```
#include <stdio.h>
void demo()
{
    auto int autovar = 0;
    static int staticvar = 0;
    printf("auto = %d, static = %d\n",
           autovar,staticvar);

    autovar++;
    staticvar++;
}
```

```
main()
{
    int i=0;
    while (i<3)
    {
        demo();
        i++;
    }
}
```

Output:

```
auto = 0, static = 0
auto = 0, static = 1
auto = 0, static = 2
```


OPERADORES DE BAIXO NÍVEL

Operadores bit a bit

&	and <i>bit a bit</i>
	or
^	xor
~	not (<i>inversão/complemento</i>)
<<	deslocamento (shift) à esquerda
>>	deslocamento (shift) à direita

OPERADORES BIT A BIT

A	B	A & B	A B	A ^ B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

OPERAÇÕES BIT A BIT

5 << 1 00000000 00000101 = 5
 00000000 00001010 = 10

255 >> 3 00000000 11111111 = 255
 00000000 00011111 = 31

$x \ll y \iff x * 2^y$

$x \gg y \iff x / 2^y$

OPERAÇÕES BIT A BIT - EXEMPLOS

$7 \ll 1$	: $111 \ll 1 = 1110 = 14$
$7 \ll 2$	: $111 \ll 2 = 11100 = 28$
$7 \gg 1$	: $111 \gg 1 = 11 = 3$
$7 \gg 3$	: $111 \gg 3 = 0$
$a \gg 1$	: $a = a \gg 1$
$7 \gg 1 \ll 1$	: $3 \ll 1 = 11 \ll 1 = 110 = 6$
$7 \& \sim 1$	: $7 \& 0 = 111 \& 0 = 110 = 6$
$a \& = \sim 1$	se $a = 7$; $a = a \& 0 = 6$
$6 \wedge 1$	: $110 \wedge 001 = 111 = 7$
$a \wedge = 1$	: se $a = 7$; $a = a \wedge 1 = 111 \wedge 001 = 110 = 6$
$a = 1$	: se $a = 7$; $a = a 1 = 111 001 = 111$
$6 1$	: $110 001 = 111 = 7$
$6 \& 15 1$	: $0110 \& 1111 = 0110 0001 = 0111 = 7$
$6 \& (15 1)$	: $0110 0001 = 0110 \& 0111 = 0110 = 6$

$$\begin{array}{r} 111 \\ \& 110 \\ \hline 110 \end{array}$$

OPERAÇÕES BIT A BIT - EXEMPLOS

/* Visualização dos valores de um Byte, com apenas um bit a 1, movimentando o bit activo da esquerda para a direita*/

0xFF	1111 1111
0x80	1000 0000
	&1000 0000

```
unsigned char ch = 0xFF & 0x80 /* Mascara para obter o 1ºbit */  
int i;  
for (i=0;i<8;i++)  
    printf("i=%d: valor= %d \n",i,ch>>i);
```


OPERAÇÕES BIT A BIT - EXEMPLOS

/* Visualização dos bits de um valor Inteiro */

```
pos = sizeof(int) * 8;  
while (--pos >= 0)  
{  
    if (x & (1 << pos)) printf("1");  
    else printf("0");  
}
```

OPERAÇÕES BIT A BIT - EXEMPLOS

/* Conta o número de bits com valor 1 */

```
int bitcount ( int x )  
{  
    int b;  
  
    for (b=0; x != 0; x >>=1)  
        if (x & 1) b++;  
    return b;  
}
```

PONTEIROS PARA FUNÇÕES

É possível enviar para uma função o endereço de outra função, porque sempre que um programa é colocado a executar, também as funções estão carregadas em memória, estando bem definido o local (endereço) onde essa função existe.

PONTEIROS PARA FUNÇÕES

□ Sintaxe:

tipo (ptr_para_função) (parâmetros da função)*

PONTEIROS PARA FUNÇÕES-EXEMPLO

```
#include <stdio.h>
```

```
int Quadrado(int x)
{ return x*x; }
```

```
int Dobro(int x)
{ return 2*x; }
```

```
int Cubo(int x)
{ return Quadrado(x)*x; }
```

```
void Imprime_Especial(
    int (*funcao)(int), int n)
{
    printf("| %d | \n",
        (*funcao)(n) );
}

main()
{
    Imprime_Especial(Quadrado,3);
    Imprime_Especial(Dobro,10);
    Imprime_Especial(Cubo,5);
}
```


FICHEIROS

❑ `FILE *p;` $\Rightarrow$ *p* é um ponteiro para um ficheiro.

- ❑ Antes de usar o ponteiro, tem de ser associado com o ficheiro que pretendemos utilizar.

*fopen(char *nome_do_ficheiro, char *modo)*

FICHEIROS

MODOS Significado

- ❑ **"r"** Abertura de um ficheiro para leitura. A leitura inicia-se no início do ficheiro. (*Insucesso fopen devolve NULL*)
- ❑ **"w"** Cria um novo ficheiro para escrita. Se o ficheiro já existe é destruído. O ponteiro indica o início do ficheiro.
- ❑ **"a"** Abre um ficheiro existente para acrescentar informação. Se não existir cria-o.
- ❑ **"r+"** Abre um ficheiro existente para leitura e escrita. Se não existir cria-o. Se existir os dados são colocados por cima dos dados anteriores.
- ❑ **"w+"** Cria um novo ficheiro para leitura ou escrita.
- ❑ **"a+"** Abre um ficheiro existente ou cria novo para acrescentar no final.
- ❑ Por defeito, a abertura de um ficheiro é realizada considerando que este é um ficheiro de texto. Para a utilização com ficheiros binários acrescentar "b" ao modo: **"rb"**

LEITURA/ESCRITA EM FICHEIROS

- ❑ ***getc()*** : Macro para a leitura de um caracter do ficheiro.

*int getc(FILE *f)*

- ❑ ***fgetc()*** : Mesmo que *getc()*, mas implementado com uma função.

- ❑ ***putc()*** : Macro para escrita de um caracter no ficheiro.

*int putc(int ch, FILE *f)*

- ❑ ***fputc()*** : Mesmo que *putc()*, mas implementado com uma função.

MANIPULAÇÃO DE FICHEIROS

- ❑ ***fclose()*** : fechar um ficheiro.

*int fclose(FILE *f)*

- ❑ ***feof()*** : determina se é fim do ficheiro. Valor diferente de zero se é EOF.

*int feof(FILE *f)*

LEITURA/ESCRITA CARACTER A CARACTER DO FICHEIRO

```
int copyfile(char *infile,char *outfile)
{
    FILE *fp1,*fp2;
    if (( fp1 = fopen(infile, 'rb')) ==NULL)
        return 0; /* Erro */
    if (( fp2 = fopen(outfile, 'wb')) ==NULL)
    {
        fclose(fp1);
        return 0;
    }
    while (!feof(fp1))
        putc(getc(fp1),fp2);
    fclose(fp1);
    fclose(fp2);
    return 1;          /* Sucesso */
}
```


FICHEIRO DE TEXTO - EXERCÍCIOS

- ❑ Fazer a leitura de um ficheiro de texto e mostrar no écran.

/* Abertura e teste do ficheiro*/

```
#include <stdio.h>
main()
{
    FILE *fp;
    char s[100];
    puts("Int. o Nome do ficheiro : ");
    gets(s);
    fp=fopen(s,"r");
    if (fp=NULL) printf("\n Impossível abrir o ficheiro !!!\n");
    else {
        printf("Ficheiro aberto com sucesso ...\n");
    }
}
```

FICHEIRO DE TEXTO - EXERCÍCIOS (CONT.)

/* Escrever no écran e utilização da constante EOF (End-of-File) */

```
char ch;  
while ( ( ch=fgetc(fp) ) != EOF )  
    putchar(ch);  
fclose(fp);  
}
```

LEITURA/ESCRITA LINHA A LINHA DO FICHEIRO

- ❑ ***fgets()***: *char *fgets(char *s, int n, FILE stream);*
 - ❑ lê um conjunto de caracteres de ***stream***, para um *array* apontado por ***s***, até que ***n-1*** caracteres sejam lidos, ou seja lido o caractere de mudança de linha e transferidos para ***s***.

EXEMPLO LEITURA/ESCRITA LINHA A LINHA

```
int copyfile( char *infile,char *outfile)
{
    FILE *fp1,*fp2;
    char line[MAXLINHA];

    if ((fp1 = fopen(infile,"r")) == NULL)
        return 0;
    if ((fp2 = fopen(outfile,"w")) == NULL)
    {
        fclose(fp1);
        return 0;
    }
    while (fgets(line,MAXLINHA-1,fp1) != NULL)
        fputs(line,fp2);
    fclose(fp1);
    fclose(fp2);
    return 1;
}
```

EXEMPLO LEITURA/ESCRITA DE UM BLOCO

```
#define BLOCKSIZE 512
typedef char DATA;
int copyfile( char *infile,char *outfile)
{
    FILE *fp1,*fp2;
    DATA bloco[BLOCKSIZE];
    int num_read;
    if ((fp1 = fopen(infile,"r")) == NULL)
        return 0;
    if ((fp2 = fopen(outfile,"w")) == NULL)
    {
        fclose(fp1);
        return 0;
    }
    while ((num_read = fread(bloco, sizeof(DATA), BLOCKSIZE,fp1)) == BLOCKSIZE)
        fwrite(bloco,sizeof(DATA),num_read, fp2);
    fwrite(bloco,sizeof(DATA),num_read, fp2);
    fclose(fp1);
    fclose(fp2);
    return 1;
}
```


INPUT E OUTPUT FORMATADO

- ❑ É possível ler ou escrever dados de forma formatada em ficheiros através das funções:
 - ❑ ***fscanf***
 - ❑ ***fprintf***
- ❑ Estas funcionam da mesma forma que as funções *scanf* e *printf*, tendo apenas mais um parâmetro inicial que corresponde ao ficheiro onde o processamento irá ser realizado.
- ❑ As funções que permitem ler ou escrever dados de forma formatada em ficheiros devem apenas ser utilizadas para ficheiros em modo de texto.

FICHEIROS *RANDOM ACCESS*

- ❑ ***fseek()***: `int fseek(FILE *f, long int offset, int origem);`
 - ❑ posiciona o ponteiro para leitura ou escrita no ficheiro ***f***. A nova posição é dada pelo valor de ***offset/salto*** em *bytes* a partir do início, ou da posição corrente, ou do fim do ficheiro, dependendo da posição indicada por ***origem***.
 - ❑ ***SEEK_SET***: início do ficheiro
 - ❑ ***SEEK_CUR***: posição corrente
 - ❑ ***SEEK_END***: fim do ficheiro

FICHEIROS *RANDOM ACCESS*

Exemplo:

```
stat= fseek(fp,10,SEEK_SET);
```

- ❑ move o indicador do ficheiro 10 caracteres no ficheiro;
- ❑ Se a operação foi feita sem problemas o valor devolvido é **zero**, se a operação é ilegal devolve um valor **diferente de zero**.
- ❑ ***ftell()*** : devolve a posição corrente do ficheiro.

ACESSO SEQUENCIAL E ACESSO DIRECTO A FICHEIROS

- ❑ O acesso à informação pode ser realizado de duas formas diferentes:
 - ❑ **Acesso Sequencial:** percorrendo o ficheiro até localizarmos aquilo que pretendemos.
 - ❑ **Acesso Directo:** Colocando-nos na posição que pretendemos de um ficheiro sem ter que percorrer todos os *bytes* que se encontram antes dessa posição.
- ❑ Só faz sentido falar em posicionamento em ficheiros quando estamos a processar ficheiros binários.

EXEMPLO

```
typedef struct  
{  
    int num;  
    char nome[45];  
} DADOS;
```

```
int i,j;  
FILE *f;  
DADOS r;  
  
/* cria 10 registos */  
f=fopen("exemplo.dat","w");  
for (i=1;i<=10; i++)  
{  
    r.num=i;  
    fwrite(&r,sizeof(DADOS),1,f);  
}  
fclose(f);
```


EXEMPLO

```
/* lê 10 registos */  
  
f=fopen("exemplo.dat","r");  
for (i=1;i<=10; i++)  
{  
    fread(&r,sizeof(DADOS),1,f);  
    printf("%d\n",r.num);  
}  
fclose(f);
```

EXEMPLO

/* usa fseek para ler por ordem inversa */

```
f=fopen("exemplo.dat","r");  
for (i=9; i>=0; i--)  
{  
    fseek(f,sizeof(DADOS)*i,SEEK_SET);  
    fread(&r,sizeof(DADOS),1,f);  
    printf("%d\n",r.num);  
}  
fclose(f);
```

Exemplo

/* Usa fseek para ler o 4º registo, altera-o, e escreve novo valor*/

```
f=fopen("exemplo.dat","r+");  
fseek(f,sizeof(DADOS)*3,SEEK_SET);  
fread(&r,sizeof(DADOS),1,f);
```

```
r.num=100;  
fseek(f,sizeof(DADOS)*3,SEEK_SET);  
fwrite(&r,sizeof(DADOS),1,f);  
fclose(f);
```